END TO END SENTIMENT ANALYSIS SYSTEM
USING NATURAL LANGUAGE PROCESSING (NLP) AND MACHINE LEARNING (ML)
FINAL REPORT – FLIPKART E-COMMERCE DATASET PROJECT
Code repository: https://github.com/henrykwong-ai-ds-ml/FlipKartSentimentAnalysis

EXECUTIVE SUMMARY

This project builds a complete end-to-end sentiment analysis pipeline for customer product reviews from a Flipkart dataset obtained from the Kaggle website. It covers data loading, Natural Language Processing (NLP), feature engineering (Bag-of-Words and Term Frequency – Inverse Document Frequency), model training and evaluation, model persistence and two machine learning (ML) models – Naïve Bayes and Logistic Regression - to classify text (such as product reviews) into three categories: positive, negative, or neutral. The project also builds a graphical user interface (GUI) using Tkinter to display side-by-side sentiment predictions from the two ML models. It is designed as a clean, reproducible template for sentiment analysis projects in Python.

The final TF-IDF + Logistic Regression model achieves 94% accuracy and strong per-class F1 scores (0.89 negative, 0.51 neutral, 0.97 positive) on a large set, outperforming Naive Bayes baselines and providing reliable, production-ready sentiment predictions for real-world e-commerce data.

The Scale: We engineered a pipeline capable of processing **205,052 e-commerce records**.

The Innovation: We combined separate text fields (Review + Summary) to optimize performance and prevent memory issues.

The Progress: We successfully boosted baseline accuracy from **92% to 94%** by swapping Naive Bayes for an optimized Logistic Regression engine.

The Deployment: We built a functional **Tkinter GUI** that translates our machine learning vectors into human-readable visual feedback (emojis).

PROJECT OVERVIEW

- Objective: Build an end-to-end sentiment analysis pipeline that processes real Flipkart product reviews and classifies them as negative, neutral, or positive, then expose the model via a Python Tkinter GUI.
- Dataset: ~205,000 rows, 6 columns: productname, productprice, Rate, Review, Summary, Sentiment, with 3 sentiment classes and class imbalance (positive dominant).

- Scope: For efficient experimentation, the core modelling phase uses a 25,000-row subset, then the final Logistic Regression model is trained and evaluated on the full cleaned dataset (~41,000 rows after mapping and filtering).

DATA PREPARATION

- Text fields “Review” and “Summary” are concatenated into a single combined text feature to maximize context and reduce pipeline complexity.
- A custom preprocessing function applies: HTML stripping, punctuation removal, lowercasing, tokenization, stopword removal using NLTK, and lemmatization via WordNetLemmatizer, with stopwords and lemmatizer preloaded for speed. We applied the WordNetLemmatizer with an explicit verb context tag (pos='v') to reduce words to their authentic base forms (e.g., "cooling" to "cool").
- For the subset experiment, only the first 25,000 rows are preprocessed and used for Naive Bayes baselines; for the final model, the entire dataset is cleaned, missing text filled, and combined text recomputed.
- Target labels are explicitly mapped from strings to integers: positive → 1, neutral → 0, negative → −1, and rows with unmapped or invalid labels are dropped to keep the supervised target clean.

MODELLING APPROACH

Baseline: Multinomial Naive Bayes on Bag-of-Words

- Vectorization: CountVectorizer with max_features=5000 and ngram_range=(1, 3) over the preprocessed combined text for the 25,000-row subset.
- Model: MultinomialNB trained on the subset; a separate train/test split (80/20) is used for honest evaluation with its own CountVectorizer fitted on train only.

Variant: Multinomial Naive Bayes on TF-IDF

- Vectorization: TfidfVectorizer with max_features=5000 and ngram_range=(1, 3), again on the 25,000-row subset.
- Model: MultinomialNB retrained on TF-IDF features, evaluated with an 80/20 split using the same random_state for alignment.

Final model: Logistic Regression on TF-IDF (full dataset)

- Data: Full Flipkart dataset of over 205,000 rows, after dropping rows with unmapped sentiment labels; Review and Summary are filled, concatenated, and used as raw text.
- Vectorization: TfidfVectorizer with max_features=5000 and ngram_range=(1, 2), fitted on the training split and reused consistently for evaluation and deployment.
- Model: LogisticRegression with max_iter set to 1000 trained on the TF-IDF representation; the target uses the clean numeric mapping (−1, 0, 1).
- Persistence: The final Logistic Regression model and the TF-IDF vectorizer are serialized to logisticregressionmodel.pkl and vectorizer.pkl, replacing earlier Naive Bayes artifacts.

EVALUATION AND RESULTS

Machine Learning Model Performance Comparison

Models	Class/Label	Precision	Recall	F1 Score	Overall Accuracy
Model 1: Naïve Bayes CountVectorizer	Negative	0.78	0.85	0.82	**0.92**
	Neutral	0.48	0.36	0.41	
	Positive	0.96	0.96	0.96	
Model 2: Naïve Bayes TF-IDF Vectorizer	Negative	0.79	0.82	0.81	**0.92**
	Neutral	0.82	0.18	0.30	
	Positive	0.94	0.98	0.96	
Model 3: Logistic Regression TF-IDF Vectorizer	Negative	0.88	0.90	0.89	**0.94**
	Neutral	0.72	0.4	0.51	
	Positive	0.96	0.98	0.97	

Naive Bayes (CountVectorizer baseline, 25k subset)

- Test split size: 5,000 samples (from 25,000 preprocessed reviews).
- Reported metrics (per class):
    - Negative: precision 0.78, recall 0.85, F1 0.82 (support 706).
    - Neutral: precision 0.48, recall 0.36, F1 0.41 (support 230).
    - Positive: precision 0.96, recall 0.96, F1 0.96 (support 4064).

- Overall accuracy: strong performance on the majority positive class and weaker performance on the minority neutral class. The Naive Bayes classification report reveals that while the model excels at identifying positive sentiment (96% recall), it is highly conservative and inaccurate when evaluating neutral statements (36% recall) due to a severe data imbalance where positive reviews overpower the baseline statistical probabilities.

Sentiment Class	Precision	Recall	F1-Score	Support
Negative	0.78	0.85	0.82	706
Neutral	0.48	0.36	0.41	230
Positive	0.96	0.96	0.96	4,064

Naive Bayes (TF-IDF variant, 25k subset)

- Using TF-IDF features shifts the class-wise F1 distribution but still underperforms on the neutral class.
- Reported metrics:
    - Negative: precision 0.79, recall 0.82, F1 0.81 (support 706).
    - Neutral: precision 0.82, recall 0.18, F1 0.30 (support 230).
    - Positive: precision 0.94, recall 0.98, F1 0.96 (support 4064).

Overall accuracy remains 0.92 indicating no overall improvement versus CountVectorizer and degrading neutral recall in particular.

Sentiment Class	Precision	Recall	F1-Score	Support
Negative	0.81	0.65	0.72	706

Sentiment Class	Precision	Recall	F1-Score	Support
Neutral	0.60	0.18	0.28	230
Positive	0.93	0.98	0.95	4,064

Logistic Regression (TF-IDF, final model)

- Train/test split: 80/20 on the full cleaned dataset, with roughly 41,011 labelled samples in the evaluation classification report (20% test).
- Reported metrics (per class):
    - Negative: precision 0.88, recall 0.90, F1 0.89 (support 5743).
    - Neutral: precision 0.72, recall 0.40, F1 0.51 (support 2013).
    - Positive: precision 0.96, recall 0.98, F1 0.97 (support 33,255).
- Overall accuracy: 0.94 representing a clear improvement over both Naive Bayes setups, especially on minority negative and neutral classes.
- A Logistic Regression confusion matrix shows cleaner separation of negative and positive, with remaining errors concentrated in neutral, which is expected given its lower support and semantic ambiguity.
- An F1-score comparison plot shows Logistic Regression improving F1 for all three classes relative to the baseline Naive Bayes: approximately 0.89 versus 0.82 for negative, 0.51 versus 0.41 for neutral, and 0.97 versus 0.96 for positive.

Sentiment Class	Precision	Recall	F1-Score	Support
Negative	0.88	0.90	0.89	5,743
Neutral	0.72	0.40	0.51	2,013

Sentiment Class	Precision	Recall	F1-Score	Support
Positive	0.96	0.98	0.97	33,255
Global Accuracy			**94%**	**41,011**

Model selection

- Based on higher overall accuracy, better macro and weighted F1, and improved per-class F1, Logistic Regression with TF-IDF was selected as the final production model.
- The TF-IDF plus Logistic Regression configuration also generalizes better to the full dataset scale and integrates cleanly into the GUI, using the same serialized vectorizer for both evaluation and runtime prediction.

DEPLOYMENT AND APPLICATION

- A Tkinter-based desktop GUI was built to serve and run inference with the trained models and display side-by-side predictions from Naive Bayes and Logistic Regression for any selected review.
- The GUI loads the saved logisticregressionmodel.pkl and vectorizer.pkl, reconstructs the same combined-input text (Review plus Summary) used at training time, and applies the TF-IDF transformation before predicting.
- Predictions are mapped back from integers (−1, 0, 1) to human-readable labels (Negative, Neutral, Positive) and accompanied by emoji icons and the original review text, providing an intuitive visual dashboard.
- Navigation controls (Next and Previous Review) allow stepping through the dataset to inspect disagreements between Naive Bayes and Logistic Regression, effectively turning the interface into a lightweight model-inspection tool.
- The interface features a dual-column workspace displaying real-time side-by-side inference outputs from both models simultaneously. Native operating system emoji lookups are dynamically bound to numerical prediction vectors, providing immediate visual confirmation of classified user feedback (😊 for Positive, 😐 for Neutral, and 😠 for Negative).

KEY TAKEAWAYS AND FUTURE WORK

- Combining “Review and Summary” into a single preprocessed text feature substantially simplifies the pipeline and likely improves context coverage for sentiment decisions.
- Naive Bayes with high-dimensional text features is a strong baseline but struggles on the minority neutral class, especially under class imbalance and more nuanced language.
- Logistic Regression with TF-IDF features and clean numeric label mapping delivers the best tradeoff between global accuracy and per-class performance, making it the preferred production model and the main inference engine for our pipeline workloads.
- E-commerce reviews are heavily skewed towards positive remarks. Future improvements could include class-weighted training or focal loss to further boost neutral performance, hyperparameter tuning of C and regularization for Logistic Regression, and experimentation with more expressive models such as linear SVMs or transformer-based encoders while keeping the existing TF-IDF and GUI integration pattern.

Explanatory Notes to Changes to the Project Architecture and Feature Engineering

We switched from **Gaussian Naive Bayes (GaussianNB)** to **Multinomial Naive Bayes (MultinomialNB)** for two major reasons: data structure compatibility and computational efficiency.

1. Handling Count Data (Discrete vs. Continuous)

Our text cleaning pipeline uses CountVectorizer(max_features=5000, ngram_range=(1, 3)). This converts the reviews into a matrix of **word frequencies and counts** (discrete integer values like 0, 1, 5, etc.).

- **GaussianNB** assumes that the features follow a normal (Gaussian) distribution. It is designed for continuous, real-valued data (like temperature, height, or stock prices). It doesn't make statistical sense for word frequencies.
- **MultinomialNB** is specifically modeled for multinomially distributed data. It is the textbook standard for text classification because it calculates probabilities based on how frequently specific words (or n-grams) appear within a given document class.

2. Memory Efficiency (Sparse vs. Dense Matrices)

The output of the CountVectorizer is a **Sparse Matrix** (a memory-efficient format that only stores the locations of words that actually appear, ignoring all the zeros).

- GaussianNB requires a **dense matrix**. To use it, we have to convert our data using .toarray(). For a vocabulary of 5,000 features across 25,000 rows, forcing a dense matrix would consume massive amounts of RAM and could easily crash our Jupyter kernel.
- **MultinomialNB** natively accepts sparse matrices. It loops through our 25,000 rows instantly without expanding the zeros in memory, allowing our training step to finish in fractions of a second.

Why I combined two columns into one: Review and Summary

To maximize semantic richness and contextual understanding, the Review (heading phrase) and Summary (full detail text) columns were consolidated into a single text stream variable before tokenization. This prevents single-input constraints, accelerates feature execution by running the preprocessing pipeline once per row instead of twice, and bypasses memory-expensive horizontal matrix alignment operations.

The purpose of doing this comes down to three main reasons:

1. Maximizing Context and Feature Richness

In sentiment analysis, more relevant context leads to better predictions.

- The **Summary** column usually contains short, high-impact emotional words (e.g., *"Amazing product!"*, *"Worst purchase ever"*).
- The **Review** column contains the deeper context, specific details, and qualifiers (e.g., *"The battery life is great, but the screen came scratched"*).

By merging them into a single string (e.g., "Amazing product! The battery life is great..."), we ensure that your CountVectorizer captures the vocabulary and n-grams from **both** fields, giving the Naive Bayes model a much richer pool of features to learn from.

2. Streamlining the NLP Preprocessing Pipeline

If I kept the columns separate, I would have to run our heavy preprocessing function (preprocessed_review) twice—once for the reviews and once for the summaries.

By concatenating them into a single combined_series using pandas, I only have to apply the cleaning function (HTML removal, punctuation stripping, lowercasing, and lemmatization) **once**. This makes the preprocessing step significantly faster and keeps our code much cleaner.

3. Adapting to a Single-Input Model Architecture

Standard text vectorizers like CountVectorizer or TfidfVectorizer expect a single sequence of text per row (a 1D array or list of strings) to construct their vocabulary matrix.

If you I left them as two separate columns, I would have to vectorize them independently and then use a function like hstack to stitch the two resulting sparse matrices together. Combining the columns at the dataframe level completely bypasses that extra matrix manipulation, allowing us to feed a single text variable (preprocessed_reviews) straight into cv.fit_transform().

3. Why Logistic Regression is Included

Logistic Regression was introduced as a second model into the project to provide a benchmark for performance and accuracy. In Natural Language Processing (NLP) tasks like sentiment analysis, Logistic Regression is an industry-standard baseline because it is highly efficient, scales well with large datasets, and handles high-dimensional text data exceptionally well.

By comparing the two, the notebook demonstrates a classic data science workflow: evaluating a simpler probabilistic model (Naive Bayes, which achieved 92% accuracy) against a linear classification model (Logistic Regression, which pushed accuracy up to 94%). This inclusion proves that optimizing text representation with TfidfVectorizer and switching to a logistic regression model yields stronger precision and recall, establishing a clear path for upgrading the Tkinter dashboard's underlying engine.

Tough Questions for Students

1. **Why did we choose the Naive Bayes classifier for this sentiment analysis project? What makes it suitable for text classification?**

Since we are evaluating Flipkart comments and trying to separate text into three categories, Naïve Bayes is a proven classifier for this project. Very fast training and prediction make it practical for sentiment analysis projects, it works well with high dimensional data such as text classification, and represents a good baseline classifier. It is also well suited to evaluate reviews via word counts or term frequency (TF) or inverse document frequency (IDF). A high IDF score indicates a word is highly relevant to a specific document. Importantly, Naïve Bayes transforms text into number and probabilities to predict which sentiment class is most likely. It is a useful benchmark before trying more complex models such as logistic regression, random forest, support vector machines or deep learning.

2. Explain the significance of preprocessing steps like lemmatization, stopword removal, and tokenization. How would the model's performance change without these steps?

Preprocessing is absolutely crucial to dealing with raw text data which is incredibly messy and complex. People write reviews and comments which contain a great deal of “noise” such as irrelevant word variations, poor punctuation and meaningless vocabulary. Tokenization, stopword removal and lemmatization act as a pipeline to clean, compress and standardize text into more data useful components.

a. Lemmatization, or standardizing word roots, reduces a word to its base or dictionary form, while taking its context and grammatical part-of-speech into account. Variations are consolidated into a single feature, ensuring the model pools all relevant sentiments together.
b. Stopword removal filters out frequently occurring words in reviews or comments such as “is”, “the”, “a”, “in” “and”, “of” and so forth that carry very little meaning or sentiment. In our dataset of over 205,000 product reviews, these words will appear millions of times regardless of the sentiments expressed in those reviews. They are widespread but add zero predictive value. Removing them shrinks the size of the feature space (the vocabulary size), thereby saving memory and speeding up model training.
c. Tokenization, or breaking up text in the form of a sentence or paragraph, into units called tokens, allows us to work with text as numerical matrices in a bag of words or tf-idf vectorizer.

Without these preprocessing steps, our Naïve Bayes model performance would degrade in significant ways:

a. Overfitting due to “feature explosion”. Punctuation marks would remain attached to words and word variations would proliferate. The same or similar root words would be treated as separate features instead of one unified indicator or sentiment. A massive feature space paired with sparse data causes the model to overfit, memorizing specific, rare string variations in the training data rather than learning generalized sentiment.
b. Excessive stopwords will skew probabilities as Naïve Bayes model calculates the probability of a class based on multiplying word likelihoods together. For example, if a positive review happens to contain five instances of the word *"the"*, the model's math can get heavily biased by the sheer volume of these neutral words, leading to misclassifications.
c. Sparsity Issues and the inability to generalize on new data. Without lemmatiztion, the model may not recognize that similar words may share the same functional context. The result will be that a model fails to recognize the sentiment of words it hasn't explicitly seen in that exact form during training, severely dropping its accuracy and recall on a real-world testing set.
d. In the Flipkart dataset we are using, the matrix generated from raw text would be unnecessarily massive. This would cause our code to consume significantly more RAM and the training/prediction phases of our application would slow down, defeating one of the main advantages of taking advantage of the speed of a Naïve Bayes classifier in the first place.

3. **What role does CountVectorizer play in the project? How would the results differ if we used TfidfVectorizer instead?**

CountVectorizer serves as a bridge to convert raw text data into the mathematics or numerical features that the Naïve Bayes classifier can use to predict sentiment. Implementing a Bag of Words model, it builds a vocabulary by scanning the entire relevant text columns of our dataset, specifically the “Review” and “Summary” columns and compiles a master list of every unique word across all records. It then constructs the matrix (vectorization) transforming each individual review into a row in a massive matrix. The columns of this matrix represent the unique words in its vocabulary. The vectorizer then looks at each review, and populates the matrix cells with raw integers representing the frequency of a given word in a specific text. CountVectorizer is useful when you want a straightforward bag-of-words representation. It preserves raw frequency information, which works well with classifiers like Naive Bayes because the model can directly

learn which words tend to appear in positive or negative reviews. It is also simple and easy to interpret, which is helpful in a project report or baseline model.

If we use the TfidfVectorizer instead, the text would still become numbers, but the numbers would be weighted by how important each word is across the whole dataset, not just how often it appears in one review. That usually reduces the influence of very common words and gives more weight to terms that help distinguish one review from another.

With TfidfVectorizer, frequent but less informative words get down weighted, so the model may focus more on distinctive sentiment terms. That can improve performance in some text classification tasks, especially when common words dominate the data. In practice, the result may be slightly better accuracy or better generalization, but the improvement depends on the dataset and classifier.

Furthermore, product reviews get fair treatment regardless of length under TfidfVectorizer so everything is evaluated on a level playing field, preventing the length of a review from skewing the sentiment prediction.

Generally, switching to TfidfVectorizer yields a noticeable boost in overall accuracy and F1-scores for Naive Bayes text classifiers. By automatically down-weighting non-informative words, the model makes fewer errors on borderline cases (like neutral or subtly sarcastic reviews).

We also see a reduction in false positives or negatives, because TfidfVectorizer isolates specific vocabulary, helping the model better distinguish between standard positive reviews and highly passionate notes, reducing misclassifications caused by generic word repetition.

For a sentiment dataset like Flipkart reviews, CountVectorizer is a good starting point, while TfidfVectorizer is the next logical comparison if we want to test whether weighting improves classification.

4. **How does the Bag-of-Words model handle unseen words in the test data that were not present in the training data?**

The Bag-of-Words model will completely ignore unseen words in the test data because it was not learned during training. Because unseen words are dropped during vectorization, they never make it into the final mathematical calculation. The Naive Bayes model computes probabilities using only the words that survived the filter. If unseen words carry important sentiment or domain specific meaning, this can lead to inaccurate or neutral predictions.

5. **Discuss the limitations of using n-grams in text vectorization. How do n-grams impact model accuracy and computational complexity?**

N-grams are useful because they capture short word sequences like “not good,” but they also create a bigger and sparser feature space as n increases. That usually means a trade-off: better local context, but more data sparsity and more computation.

Main limitations

- Data sparsity increases quickly with larger n, so many valid n-grams may never appear in training.
- They only capture local context, so they miss long-range meaning and broader sentence structure.
- They have weak semantic understanding, so different words with similar meaning are treated as unrelated symbols.
- The feature space grows exponentially, which increases memory use and training cost.

Effect on accuracy: Adding n-grams can improve accuracy when the sequence matters, especially in sentiment analysis where phrases like “not good” are more informative than single words. But if you keep increasing n, the model often gets worse because many features become rare or unseen, which makes learning less reliable. In practice, bigrams or trigrams often help more than very large n-grams.

Effect on complexity: Higher-order n-grams make the vocabulary much larger, so the model needs more memory and more time to build and use the feature matrix. That also means training and prediction become slower, especially on large datasets. So, while n-grams can improve performance, they do so at the cost of efficiency and sparsity.

To mitigate these limitations, we should apply the following constraints in our vectorizer:

- limit max features (max_features =5000) as used in our coding.
- set a minimum document frequency (e.g., min_df=3) to ignore n-grams that don’t appear in at least 2 or 3 different reviews. This cuts out unique, non-generalizable phrases.
- Stick to bigrams (although in our coding, we are using ngram_range=(1, 3)). For traditional classifiers like Naive Bayes, stopping at ngram_range=(1, 2) represents the optimal sweet spot between capturing contextual modifiers (like negations) and keeping computational complexity under control.

For our sentiment project, small n-grams like bigrams can help catch negation and common product-review phrases, but going too high usually adds noise and computational burden without much gain.

6. **What are the key differences between stemming and lemmatization? Why did we use lemmatization in this project?**

Stemming and lemmatization both reduce words to a base form, but they do it differently. Stemming is a rough, rule-based shortcut that strips prefixes or suffixes, while lemmatization uses dictionary and language rules to return a valid word in its normalized form.

Key differences

- Stemming is faster but less accurate, and it can produce non-words like truncated stems that is not a real dictionary word. Stemming operates completely blindly without any understanding of English grammar, context or vocabulary.
- Lemmatization is slower but more linguistically correct, because it tries to keep the output as a real dictionary word by looking up words in a database and analyzing grammar.
- Stemming ignores context, while lemmatization can account for grammar and part of speech.
- Lemmatization generally preserves meaning better, which makes it more suitable for tasks where word sense matters.

For this project, lemmatization helps merge different forms of the same word. This process reduces feature fragmentation, lowers dimensionality, and helps the model learn stronger patterns from text.

It does a much better job at recognizing how words point to a particular sentiment, whether it is positive, negative or neutral, thus making Naïve Bayes probability calculations much more accurate. Lemmatization is a better choice than stemming because review sentiment depends on meaningful word forms, and stemming can distort words too aggressively, which may weaken classification performance. In short, despite taking slightly longer to compute than stemming, lemmatization usually gives cleaner input and more reliable text features for sentiment models. Naïve Bayes trains very fast so any extra preprocessing time is a negligible trade-off for a substantial boost in accuracy and cleaner data presentation.

7. **What challenges arise when handling imbalanced datasets in sentiment analysis? How could you modify this project if the dataset were heavily imbalanced?**

It is not unreasonable to predict that in our Flipkart dataset, there will be a higher percentage of satisfied customers who will leave a large number of 5-star positive reviews, while neutral and negative reviews make up a much smaller fraction of the dataset. If this were not the case, the Flipkart e-commerce business would be in serious trouble! In machine learning and sentiment analysis, handling an imbalanced dataset – where one sentiment class heavily outnumbers others – can lead to several challenges:

a. The "Accuracy Paradox"

The most dangerous trap of an imbalanced dataset is that your model can achieve an exceptionally high overall classification accuracy while being completely useless.

- The Problem: If your dataset consists of 90% positive reviews and 10% negative reviews, a completely broken model that blindly guesses "positive" for *every single input* will instantly achieve 90% accuracy.
- The Consequence: High accuracy gives you a false sense of security. The model hasn't actually learned how to identify negative customer experiences—it has just memorized the fact that the positive class is overwhelmingly common.

b. Prior Probability Bias in Naïve Bayes

The Naïve Bayes classifier is highly sensitive to the distribution of classes. During prediction, the large prior probability acts like a paperweight on the scale. Even if a test review contains strong negative words, the massive weight of the positive prior can easily overwhelm the text's evidence, forcing the model to misclassify the review as positive.

c. Poor Generalization on Minority Classes

Because the model sees very few examples of negative or neutral reviews during training, it doesn't get enough exposure to build a rich, representative vocabulary for those classes. It fails to learn the diverse ways people express disappointment or neutrality, leading to high **false-negative** rates for minority classes.

Without any imbalance handling, the model would likely predict the dominant sentiment more often and miss minority examples. After rebalancing or weighting, recall for the minority class should improve, and the model usually gives a more truthful picture of sentiment performance even if raw accuracy drops a little.

We should also use other metrics to determine sentiment accuracy such as an F1 score or confusion matrix. F1 is useful because it tells you whether the model is both catching positive/negative reviews and avoiding wrong predictions. This allows us to evaluate each class independently.

We can also implement resampling techniques such as balancing the dataset before it is processed by the CountVectorizer either thru under-sampling or over-sampling reviews until each class are similar in size.

We can even adjust the code parameters in Naïve Bayes to counteract the prior probability bias using a different classifier in sklearn – the MultinomialNB.

from sklearn.naive_bayes import MultinomialNB

We have gone ahead with this classifier modification in our project.

```
# Option A: Force the model to treat all classes as equally likely, ignoring the baseline dataset split
model = MultinomialNB(fit_prior=False)

# Option B: Manually input custom prior weights to balance the math
model = MultinomialNB(class_prior=[0.33, 0.33, 0.33])
```

Adjusting the parameter to fit_prior=False strips away the built-in bias toward the majority sentiment class, ensuring that all sentiment categories are treated as equally likely before any text is processed. This structural modification forces the model to judge a product review purely on the semantic evidence of the specific words written rather than an unfair baseline dataset skew.

For a Flipkart-style project, the cleanest upgrade would be to keep the same preprocessing and vectorization pipeline, then add stratified splitting, class weighting or resampling, and F1-based evaluation.

8. **Explain why we used pickle to save the model and vectorizer. What are the alternatives, and how do they compare in terms of performance and usability?**

We used **pickle** because it lets us save the trained model and vectorizer exactly as Python objects, then load them later without retraining or rebuilding the feature pipeline. That makes deployment simple: the same vectorizer vocabulary and the same fitted model state are restored for prediction.

Pickle is built into Python, so it is easy to use and adds no extra dependency. It is especially convenient for quick experiments, notebooks, and small-to-medium ML projects (like ours).

Alternatives to pickle include:

- **Joblib**: often faster and more memory-efficient for scikit-learn objects that contain large NumPy arrays. Significantly faster than pickle when dealing with large, dense numerical arrays or massive sparse matrices (like those generated from thousands of product reviews). It features highly efficiently compression algorithms to keep file sizes nice and small. For workflows involving large datasets, Joblib is generally preferred over standard pickle.

- **JSON**: useful if you want to save parameters in a human-readable form, but it does not directly save a fitted model object the way pickle does.
- **PMML(Predictive Model Markup Language)**: more portable and less tied to Python class definitions, but usually slower and more complex to set up. Moderate performance, moderate to low usability.
- **ONNX (Open Neural Network Exchange)**: Exceptional speed, but high complexity. Converting a scikit-learn model and text vectorizer into an ONNX pipeline requires specialized conversion libraries like sklearn-onnx. Best choice if you plan to migrate your model out of Python.
- **Framework-specific formats**: such as TensorFlow SavedModel or HDF5, but those are mainly for deep learning models rather than classic scikit-learn pipelines.

Pickle is the easiest option, but it is not always the most efficient or safest choice. Joblib is usually better when the model or vectorizer stores large arrays, while pickle remains the simplest for straightforward Python workflows. Also, scikit-learn warns that loading pickle files from untrusted sources can execute malicious code, so it should only be used with trusted files.

For a Flipkart sentiment analysis pipeline, pickle makes sense because we need to save both the fitted vectorizer and the trained classifier so predictions on new reviews use exactly the same vocabulary and feature mapping. In other words, pickle helps keep the training and inference steps consistent.

9. **Why is it important to split the dataset into training and test sets? What could happen if we trained the model on the entire dataset without testing?**

Splitting a dataset into distinct training and testing sets is one of the most fundamental rules of machine learning.

We want to simulate how our model will perform in the real world. The training set allows the model to learn patterns from the data and a test set lets you check whether a model can generalize to new, unseen data. If a model scores well on data it has never seen before, you can trust its predictive power.

What can go wrong without testing?

- You can get **overfitting**, where the model learns noise and details specific to the training data instead of general language patterns.
- You may think the model is performing well when it actually fails on new reviews or unseen inputs. It gives you false confidence and an illusion of success when the model appears to be performing

flawlessly based on old, previous or existing training data. However, when presented with new data, the model's accuracy degrades or plummets.

- You lose a reliable way to compare different models or settings, because there is no unbiased benchmark.

The goal of our project is to predict the sentiment of reviews the model has never seen before. If we trained on the entire dataset and never tested separately, we would not know whether the model is genuinely learning sentiment patterns or just fitting the training reviews too closely.

In short, training on all the data without a test set makes the reported performance unreliable and can hide negative real-world reviews.

10. **How does Tkinter interact with machine learning models in this project? What happens behind the scenes when a new review is selected?**

Tkinter is a graphical framework for Python to create a user interface. Tkinter acts as a presentation layer or **front end** of the application: it gives the user a simple interface to pick or type a review, while the trained Naïve Bayes or Logistic Regression model and vectorizer act as a analytical engine to make the actual prediction in the background(**back-end**). In this project, Tkinter triggers the Python code that loads the saved vectorizer and classifier, processes the text, and displays the result.

When a new review is selected, the following sequence takes place behind the scenes:

1. Tkinter captures the review text from the widget or dropdown selection.
2. The text is preprocessed the same way as training data, such as stopword removal, tokenization, cleaning, and lemmatization.
3. The saved vectorizer transforms the review into a numeric feature vector using the same vocabulary learned during training.
4. The saved model receives that vector and outputs a sentiment label or probability.
5. Tkinter displays the predicted sentiment on the screen. In our project, we are using emojis to display the predicted sentiment on screen.

   Usefulness of this setup - This separation keeps the interface simple and the ML pipeline consistent. Because the vectorizer and model are loaded from saved files, the app can make predictions without retraining each time the user clicks a review. That makes the app faster and easier to use in an interactive demo or desktop tool.

11. **In what scenarios might the sentiment prediction in this project be incorrect? What would you do to improve accuracy?**

Human language is often messy, subjective and deeply nuanced. Our model can be wrong when the review uses **sarcasm, negation, slang, spelling mistakes, or domain-specific language** that the training data did not teach it well. It can also fail on mixed reviews, where a customer praises one aspect but criticizes another, because a single overall label may not capture both sentiments. The model does not and cannot "read between the lines" and it does not handle ambiguity well. In our Flipkart dataset, it is highly likely that the model will not know how to handle local slang such as "Hinglish" – Hindi written in English script which is highly common on an Indian e-commerce platform like Flipkart. Standard English lemmatizers will fail to recognize such words and will be dropped as unseen noise leaving the model with zero context to make a prediction.

When errors happen

- Sarcastic text like "Great, another broken charger" can look positive on the surface but actually be negative.
- Negation like "not good" or "never worked" can confuse simpler word-based models if context is not handled well.
- Very short reviews may not contain enough signal for reliable prediction.
- New slang, typos, and product-specific terms may be misread if they were rare or absent in training.
- Reviews with both positive and negative aspects can be forced into one label and lose nuance.

How to improve accuracy

- Add more diverse training data, especially examples with sarcasm, negation, and tricky phrasing.
- Improve preprocessing by normalizing text, handling misspellings, and keeping negation-aware features.
- Upgrade the vectorizer to use n-grams. Use stronger features such as n-grams so the model can learn phrases like "not worth" or "very good". For example, we can use the code

  Vectorizer = TfidfVectorizer(ngram-range=(1, 2)).
- Try more advanced models, such as linear SVM, logistic regression, ensemble tree models (LightGBM or XGBoost) or transformer-based models, if you need better context understanding.
- Review misclassified samples and retrain periodically with fresh data to keep the model aligned with real review language.

For our Flipkart project, the biggest practical gains would usually come from better preprocessing, adding bigrams or trigrams, and retraining with more examples that reflect real-world review language.

12. **If you had to scale this project for a real-time application with thousands of users, what architectural changes would you recommend?**

Right now, this project lives on my desktop and will soon be uploaded to the Github repository and Google drive.

New architecture recommended (based on limited knowledge and experience with website building):

- In order to scale this project for a real-time application with thousands of users, we would have to start by creating a web application programming interface (API). No more Tkinter!
- The Naïve Bayes, Logistic Regression or other suitable sentiment analysis model, and the vectorizer would be then loaded at startup and kept in memory to avoid repeated deserialization overhead.
- Add caching for repeated or near-duplicate reviews, especially if the same text is queried often.
- Use a Docker container to package model files, the API, scikit-learn, text preprocessing tools to ensure the execution environment behaves identically regardless of where it is deployed.
- We would probably want data assets saved to the cloud so that it can be accessed by users worldwide.

We want a website application that is fast and responsive, able to handle thousands of users simultaneously with low latency. We want the vectorizer and model to be deployed together so training and generalizations stay aligned. We want a pipeline that avoids any mismatches between training time and production time text processing.

To make it reliable, I would add logging for inputs, predictions, confidence scores, and errors so we can detect drift or failures over time. I would use health checks, autoscaling, and blue-green or shadow deployments to roll out updates safely. For a real production system, this matters as much as accuracy because a fast but unstable model is not useful at scale.

13. **How would you extend this project to handle more nuanced sentiments beyond just positive, neutral, and negative (e.g., very positive, very negative)?**

To handle more nuanced sentiments, I would transform the usual 3 class setup to a 5-class sentiment model to include the two additional classes. I would label the data with the two additional categories so as to teach the model how to distinguish intensity. In terms of data, I would collect as many more examples of the new classes as possible and add strong words, intensifiers and negation patterns to train the model to gauge strength of sentiment. I would also use the "Rate" column which runs on a scale of 1 -5 in our Flipkart dataset to help the model assign the correct sentiments. For example, a rating of 1 would correspond to very negative and a rating of 5 would correspond with very positive.

I would change the basic Bag-of-Words unigram model to a TF-IDF (Term Frequency – Inverse Document Frequency) model with higher n-grams.

Lastly, in terms of Tkinter coding, we can add additional emojis or sentiment styles to reflect five categories instead of three.

14. **How does the Naive Bayes algorithm make predictions based on probabilities? What assumptions does it make, and how do they affect its performance on real-world data?**

The Naïve Bayes algorithm makes predictions by computing the probability that a text belongs to a particular class, then choosing the class with the highest value. It uses Bayes' theorem to combine the prior probability of each class (e.g., 85%) with the likelihood of the observed words, and then it picks the class with the largest posterior probability which is the final score the model computes.

Naïve Bayes makes an unrealistic conditional independence assumption (hence the name "Naïve"): it treats each feature as independent of the others once the class is known. It often assumes that all features contribute equally, even though real language is more complicated. Different variants of Naive Bayes also assume different distributions for the features, such as Multinomial for word counts or Gaussian for continuous values.

Although imperfect when used with real world data, the assumptions the model makes often performs surprisingly well because the simplified probability calculation is fast, stable, and effective on sparse text features. Naïve Bayes tends to work well on sentiment analysis because sentiment signals are often carried by individual words or short phrases, and the model is good at learning those word-class associations. Its main weaknesses with real word data include missing context, misunderstanding negation and misinterpreting word interactions.

15. **Explain the purpose of the confusion matrix and classification report in this project. What can these metrics tell us about the model's performance that accuracy alone cannot?**

The **confusion matrix** and **classification report** give a detailed, per-class view of how the model is performing, instead of just a single overall accuracy number. They show exactly which classes the model confuses and how reliable each prediction is. They are both tools that break down exactly where a model is succeeding and where it is making mistakes.

Purpose in this project

In our sentiment analysis project, the confusion matrix is a table that compares actual labels (positive/neutral/negative) to predicted labels. For our 3-class sentiment project, the matrix forms a 3 X 3 grid (See Python notebook and model output report). Each row represents the true class and each column represents the predicted class, so we can see:

- How many positive reviews were correctly classified as positive
- How many positives were misclassified as neutral or negative
- Which classes are being confused most often.
- The Diagonal (True Positives/Negatives): The cells running diagonally from the top-left to the bottom-right represent correct predictions. This is where the predicted label perfectly matches the actual label.
- The Off-Diagonal (The Errors): Any cell outside that diagonal represents a specific type of misclassification.
- The Matrix maps the exact nature of the model's confusion. For example, it will show you if the model is accidentally misclassifying severe Negative reviews (e.g., *"useless product, very bad"*) as Neutral, or if it is confusing Neutral reviews with Positive ones. This helps us understand if the model is slightly missing the mark or failing badly altogether.

The **classification report** computes metrics like **precision, recall, F1-score, and support** for each class and summarizes them. Precision measures out of all reviews the model predicted belonging to a certain class, how many were actually correct? Recall measures out of all actual reviews of a certain class in our dataset, how many did the model successfully find and catch? The F1-score measures the harmonic mean of precision and recall. It gives us a single score between 0 and 1 that represents the performance of a specific class. If either the precision or recall is inaccurate, the F1-score decreases, making it an excellent metric for spotting weaknesses. Taking together, these metrics provide us with a clear picture of the model's strengths and weaknesses per sentiment category.

What they tell you that accuracy alone cannot

Accuracy only tells you the overall proportion of correct predictions, which can be misleading, especially on imbalanced data. A model might have 80% accuracy while completely ignoring the neutral class, which accuracy alone would not reveal.

The confusion matrix and classification report let you see:

1. **Misclassification patterns**: which classes are being confused (e.g., neutral frequently predicted as positive).
2. **Per-class precision**: how many times the model is correct when it predicts a class.
3. **Per-class recall**: how well the model identifies all instances of a class.
4. **F1-score per class**: a balanced measure of precision and recall that is more informative than accuracy for imbalanced datasets.
5. **Support**: how many samples exist for each class, which helps interpret whether low performance is due to small data for a class.

Accuracy can hide class imbalance: Accuracy measures the overall percentage of correct guesses across the entire dataset. It allows a massive majority class to mask total failure on minority classes.
The confusion matrix and classification report measure real-world business impact: In an e-commerce setting, catching a negative review is often much more valuable than confirming a positive one. The classification report isolates the minority classes, allowing you to optimize your pipeline specifically to catch product defects or customer service issues.
Finally, they guide feature engineering: If the classification report reveals high precision but terrible recall for neutral reviews, it tells you that the model is too hesitant to guess "Neutral." This signals that you need to adjust your text preprocessing, modify your vectorizer settings, or balance out your dataset's class distribution.

For our Flipkart sentiment project, this means we can detect if the model is, for example, good at identifying positive reviews but poor at distinguishing neutral ones, and then adjust class weights, add data, or refine features accordingly.

Code repository: https://github.com/henrykwong-ai-ds-ml/FlipKartSentimentAnalysis